Proyecto Equilibrio Físico

Autores: Regina Soledad Noriega Ramírez y Pablo Vargas Rocha

Objetivo

Graficar en un diagrama ternario, la temperatura de rocío y burbuja para el sistema: metanol, ciclohexano y acetona.

Resolución

Se modelo con el modelo de doble phi con la ecuación de Peng-Robinson y las operaciones se hicieron en el lenguaje de programación Python con ayuda de la biblioteca Numpy para poder hacer cálculos de manera vectorizada, similar a MatLab. Ya terminado el programa, se compilo con pyinstaller a un ejecutable de Windows .exe. para que tenga la funcionalidad de poder correr en cualquier dispositivo Windows sin la necesidad de dependencias externas.

Funcionamiento

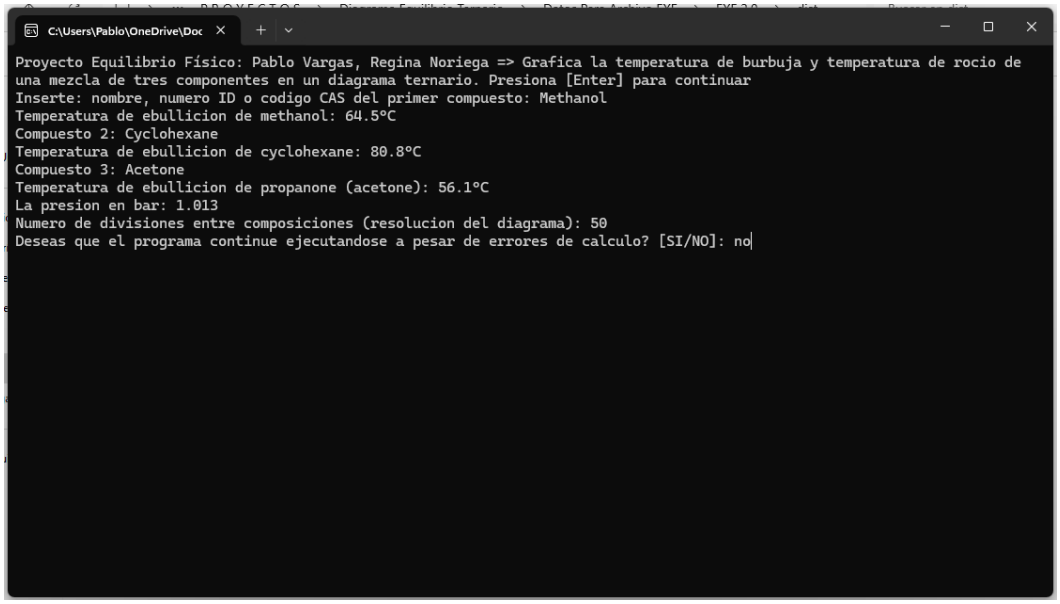
El programa puede graficar cualquier sistema de tres compuestos con las siguientes condiciones:

Cada compuesto:

1. Debe existir en el DataBank del libro “The Properties of Gases and Liquids. 5ta Edición”
2. En el DataBank deben estar disponibles las propiedades: temperatura crítica, presión critica, factor acéntrico.

Si se cumplen estas dos condiciones, el programa intentara encontrar la temperatura de burbuja y rocio empleando el modelo de doble phi. Claro, también puede fallar si los compuestos que se escogieron son inestables en el modelo de doble phi. Ej. Si se intenta modelar helio, neón y agua, pues desde luego que no va a funcionar correctamente.

Al abrir el programa, este hace una serie de preguntas para que especifiques tu sistema:



1. **“Inserte nombre, numero ID o código CAS del compuesto”**

Para identificar cada compuesto del sistema: Debes insertar el nombre del compuesto (tal como viene en el DataBank del libro, en inglés), el numero CAS o el número de fila que tiene en el DataBank.

Después te proporciona la temperatura de ebullición normal de ese compuesto. Para que te des una idea de si el sistema que quieres modelar será estable (Los compuestos deben tener temperaturas de ebullición similares).

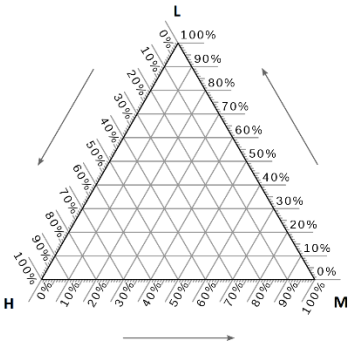
2. **La presión (en bar).**

El diagrama grafica a presión constante.

3. **Numero de divisiones entre composiciones**

El número de divisiones es cuantos pasos quieres que tome entre composiciones, lo cual se relaciona directamente con la resolución del diagrama.

Ej.



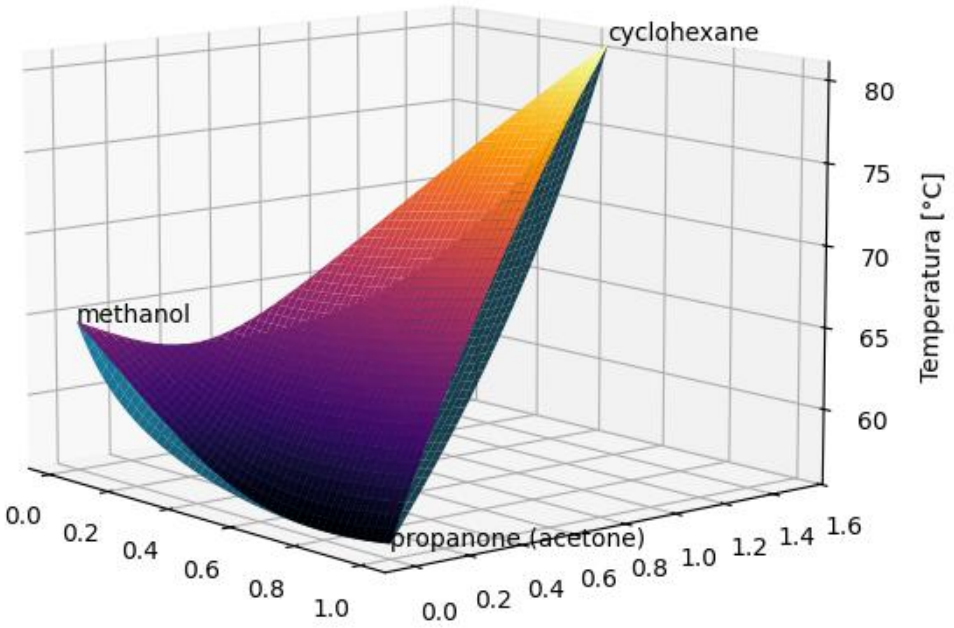
Este diagrama ternario tiene 11 divisiones entre composiciones. (11 Ya que también se cuenta el 0%)

4. **Deseas que el programa continue ejecutándose a pesar de errores de calculo**

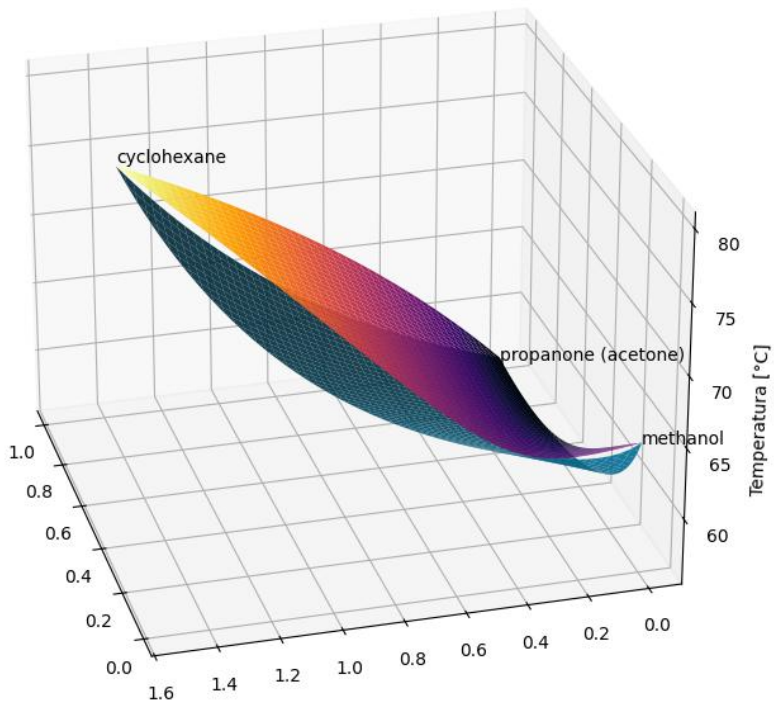
Para ciertos compuestos (como el agua), el modelo doble-ϕ puede presentar dificultades de convergencia, lo que puede provocar que no se encuentre solución en algunos puntos del diagrama ternario.
Si esta opción está habilitada, el programa ignorará esos casos puntuales y continuará intentando resolver los demás puntos. En cambio, si la opción está deshabilitada, el programa se detendrá tan pronto como falle en encontrar solución para algún punto del diagrama.

5. **Una ves se responden todas las preguntas**

El programa imprime el progreso de calculo (ya que suele tardar por arriba de un minuto, dependiendo de la resolución). Cuando se alcanza el 100%, imprime automáticamente la superficie 3D en un diagrama ternario.



Se puede observar que el modelo de doble phi no encontró un azeótropo triple. Únicamente el sistema binario de metanol con ciclohexano posee un azeótropo.



Estructura General

Antes de ser compilado, el programa funcionaba con los siguientes archivos:

funciones.py	18/05/2025 10:18 p. m.	Archivo de origen ...	19 KB
MAPA_COLUMNAS.json	05/01/2025 12:53 a. m.	Archivo de origen ...	3 KB
MAPA_ID.json	02/01/2025 09:46 p. m.	Archivo de origen ...	27 KB
PROP_TERMOF_2.1.npy	22/04/2025 01:33 a. m.	Archivo NPY	239 KB
Proyecto_Equilibrio_Fisico.py	28/05/2025 08:36 p. m.	Archivo de origen ...	12 KB

Funciones.py Contiene funciones que Proyecto_Equilibrio_Fisico.py utiliza.

Proyecto_Equilibrio_Fisico.py Contiene el script principal, el que te hace las preguntas y el encargado de resolver el equilibrio para cada punto.

MAPA_COLUMNAS.json Es un diccionario que relaciona propiedades de compuestos (ej. Temperatura critica, presion critica, etc) a una columna en la base de datos PROP_TERMOF_2.1.npy

MAPA_ID.json Es un diccionario que relaciona compuestos químicos a una fila en la base de datos PROP_TERMOF_2.1.npy

PROP_TERMOF_2.1.npy La base de datos del programa. Contiene todo el DataBank. Relaciona los compuestos con su respectiva propiedad.

```
>>> PROP_TERMOF[440, 0]
'H2O'
>>> PROP_TERMOF[66, 1]
'ethanol'
>>> PROP_TERMOF[440, 2]
'7732-18-5'
>>> |
```

Por ejemplo: La fila 440 contiene el compuesto agua, la columna 0 contiene la formula química. La fila 66 contiene el compuesto etanol, la columna 1 contiene el nombre.

Resolución del Equilibrio

Temperatura de rocío.

Para hallar la temperatura de rocio, se plantea la siguiente función error.

$$F(x_1, x_2, T) = \begin{pmatrix} x_1\phi_1^L - y_1\phi_1^v \\ x_2\phi_2^L - y_2\phi_2^v \\ x_3\phi_3^L - y_3\phi_3^v \end{pmatrix}$$

Utilizando el método de Newton Raphson (versión de cálculo vectorial), se busca un vector (x_1, x_2, T) tal que $F(x_1, x_2, T) = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$.

En la temperatura de roció; las composiciones de alimentación son iguales a las composiciones del vapor.

Temperatura de burbuja

Para hallar la temperatura de burbuja, se plantea la siguiente función error.

$$F(y_1, y_2, T) = \begin{pmatrix} x_1\phi_1^L - y_1\phi_1^v \\ x_2\phi_2^L - y_2\phi_2^v \\ x_3\phi_3^L - y_3\phi_3^v \end{pmatrix}$$

Utilizando el método de Newton Raphson (versión de cálculo vectorial). Se busca un vector (y_1, y_2, T) tal que $F(y_1, y_2, T) = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$.

En la temperatura de burbuja; las composiciones de alimentación son iguales a las composiciones del líquido.